



udp UNIVERSIDAD
DIEGO PORTALES

UNIVERSIDAD DIEGO PORTALES
ESCUELA DE INFORMÁTICA &
TELECOMUNICACIONES

ESTRUCTURAS DE DATOS & ANÁLISIS DE ALGORITMOS

Laboratorio 4: Árboles de búsqueda

Autores:

Felipe Gómez

Amaro Chiessa

Profesor:

Marcos Fantoal

14/06/2026

Índice

1. Introducción	2
2. Implementación	3
2.1. Entidades de Datos:	3
2.2. Adaptación de las Estructuras:	3
2.3. Motor de Simulación y Control:	3
3. Diseño Experimental	4
3.1. Escenarios de Carga y Trabajo:	4
3.2. Medición y Exportación de Datos:	4
3.3. Validación:	5
4. Resultados Empíricos	5
4.1. Tiempos de Ejecución y Estabilidad:	5
4.2. Análisis de Alturas Estructurales:	5
4.3. Crecimiento Asintótico:	6
5. Análisis Teórico	6
5.1. Tablas de Simbolos:	6
5.2. Árbol Binario de Búsqueda (BST):	7
5.3. Árbol Rojo-Negro (RedBlackBST):	7
6. Comparación vs Teoría	8
6.1. Crecimiento del tiempo y gráfico log-log:	8
6.2. Alturas y su Relación con el Tiempo:	8
7. Conclusión	8

1. Introducción

El Club de Robótica de la universidad administra un volumen considerable y diverso de componentes electrónicos y mecánicos, tales como sensores, motores, placas de desarrollo y herramientas. A medida que la organización crece, el flujo de transacciones semanales asociado a la compra de materiales, préstamos a estudiantes, devoluciones y bajas de elementos dañados o descontinuados se incrementa de forma sustancial. Ante este escenario, surge el problema crítico de implementar un sistema que permita almacenar, consultar, actualizar y eliminar registros con rapidez. Una gestión ineficiente del inventario puede provocar retrasos operativos y un uso excesivo de recursos computacionales. Por lo tanto, este proyecto determina experimentalmente qué estructura de datos ofrece el mejor desempeño y estabilidad para administrar un inventario dinámico sometido a una larga secuencia de operaciones.

Para resolver este problema, se diseñó un sistema automatizado de inventario donde cada componente se modela mediante pares de clave y valor. El núcleo del proyecto consiste en realizar una comparación empírica y teórica del rendimiento de dos estructuras de búsqueda específicas dadas por el texto guía de Sedgewick y Wayne, el cual sería el árbol binario de búsqueda convencional (BST) y el árbol auto-balanceado rojo-negro (RedBlackBST). La primera estructura es un árbol no necesariamente balanceado cuya forma depende del orden de inserción, mientras que la segunda aplica un conjunto de reglas internas para mantener el balance estructural. Ambas estructuras fueron integradas de manera uniforme para procesar masivamente exactamente las mismas secuencias de operaciones, permitiendo evaluar de forma directa su comportamiento en un escenario controlado.

El estudio se llevó a cabo de la siguiente manera. En primer lugar, se codificaron clases en Java para representar los artículos del inventario, las operaciones de ejecución y un módulo generador de datos aleatorios calibrado con semillas fijas para asegurar la reproducibilidad estricta de los escenarios. Posteriormente, se diseñó una interfaz común para unificar el comportamiento de los árboles y someterlos a cargas de trabajo incrementales, con tamaños de operaciones m que variaron en potencias de dos desde 4.096 hasta 524.288. Cada escenario se replicó en 30 instancias independientes para obtener significancia estadística. El tiempo de ejecución se midió de forma aislada mediante la clase `StopwatchCPU`, registrando únicamente el procesamiento interno de las operaciones y excluyendo tiempos de generación de datos o escritura. Al finalizar cada simulación, se aplicaron validaciones para garantizar la correctitud del stock y la coincidencia exacta de los contenidos en ambas estructuras. Finalmente, los resultados de tiempo, tamaño final y altura se exportaron a archivos CSV para analizar empíricamente la diferencia en el desempeño y contrastar las observaciones prácticas con la teoría Big-O.

2. Implementación

2.1. Entidades de Datos:

La clase `InventoryItem` fue desarrollada para modelar el estado de cada componente físico, encapsulando su identificador, metadata y balances de stock. Para la manipulación del inventario, se implementaron los métodos `addStock`, `lend` y `receive`. Estos incluyeron validaciones lógicas estrictas para verificar que el parámetro `itemId` coincidiera con la instancia actual, impidiendo la generación de saldos negativos o el préstamo de unidades inexistentes.

```
public boolean lend(int itemId, int quantity) {
    if (this.id == itemId && quantity > 0 && this.stockAvailable >= quantity) {
        this.stockAvailable -= quantity;
        this.stockOnLoan += quantity;
        return true;
    }
    return false;
}

public boolean receive(int itemId, int quantity) {
    if (this.id == itemId && quantity > 0 && this.stockOnLoan >= quantity) {
        this.stockAvailable += quantity;
        this.stockOnLoan -= quantity;
        return true;
    }
    return false;
}
```

Figura 1: Implementación `InventoryItem`

En cambio, la clase `InventoryOperation` actuó como un contenedor inmutable para cada transacción, almacenando el tipo de acción (dictado por el enumerador `OperationType`), la clave del componente y las cantidades involucradas en la operación.

2.2. Adaptación de las Estructuras:

Para aislar la lógica de la simulación de las complejidades subyacentes de cada árbol, se diseñó la interfaz `InventoryIndex`. Utilizando el patrón de diseño adaptador, se crearon las clases `BSTInventoryIndex` y `RedBlackBSTInventoryIndex`, las cuales envolvieron funcionalmente a las estructuras homónimas provistas por la biblioteca oficial de Sedgewick y Wayne, permitiendo que el sistema interactuara con ambas bajo el mismo contrato de métodos.

```
public interface InventoryIndex {
    void put(Integer key, InventoryItem value);
    InventoryItem get(Integer key);
    void delete(Integer key);
    boolean contains(Integer key);
    Iterable<Integer> keys();
    int size();
    int height();
}
```

Figura 2: Implementación `InventoryIndex`

2.3. Motor de Simulación y Control:

El motor de la simulación fue la clase `DataGenerator`. Esta clase construyó las características aleatorias de los componentes y orquestó la creación de las secuencias

de operaciones. Para gestionar adecuadamente los eventos, el generador implementó internamente una estructura HashSet que mantuvo un registro en tiempo real de las claves activas, permitiendo decidir de forma lógica cuándo inyectar un componente completamente nuevo o cuándo actualizar uno existente.

Finalmente, para asegurar que ambas estructuras ejecutaran de forma irrefutable la misma secuencia, la clase Experiment instanció dos listas de operaciones completamente independientes (opsBST y opsRB). Ambas listas fueron generadas utilizando la misma semilla matemática mediante StdRandom.setSeed(). Esto garantizó que, al evitar referencias cruzadas en memoria, el contenido probabilístico y el orden exacto de las operaciones a evaluar fuesen matemáticamente idénticos para el BST y el RedBlackBST.

```
for (int i = 1; i <= 30; i++) {
    long seed = m + i;

    ArrayList<InventoryOperation> opsBST = DataGenerator.generateOperations(m, keyUniverse, seed);
    ArrayList<InventoryOperation> opsRB = DataGenerator.generateOperations(m, keyUniverse, seed);

    BSTInventoryIndex bstIndex = new BSTInventoryIndex();
    RedBlackBSTInventoryIndex rbIndex = new RedBlackBSTInventoryIndex();

    Metrics bstMetrics = runOperations(bstIndex, opsBST);
    writeMetrics(writer, i, "BST", m, bstMetrics);

    Metrics rbMetrics = runOperations(rbIndex, opsRB);
    writeMetrics(writer, i, "RedBlackBST", m, rbMetrics);

    if (!validate(bstIndex, rbIndex, m, keyUniverse)) {
        System.err.println("Error de validación en tamaño " + m + ", instancia " + i);
    }
}
```

Figura 3: Implementación For anidado generador de opsBST y opsRB

3. Diseño Experimental

3.1. Escenarios de Carga y Trabajo:

El diseño experimental evaluó el comportamiento de los árboles bajo cargas masivas y crecientes de operaciones representadas por tamaños $m = 2^t$, donde t escaló desde 12 hasta 19, cubriendo un espectro desde 4,096 hasta 524,288 transacciones. Para dotar a los resultados de robustez estadística, cada tamaño se ejecutó en 30 instancias independientes. Cada repetición fijó su comportamiento calculando una semilla igual a $m + i$ (donde i es la instancia actual), limitando el universo de claves posibles a $4m$.

La distribución probabilística de eventos mantuvo estrictamente la proporción exigida por el caso de estudio: 35 % de ingresos (PURCHASE), 30 % de consultas (QUERY), 15 % de préstamos (LEND), 10 % de devoluciones (RECEIVE) y 10 % de bajas por obsolescencia (DISPOSE).

3.2. Medición y Exportación de Datos:

La medición empírica del desempeño se acotó meticulosamente mediante la clase StopwatchCPU. El cronómetro fue iniciado exclusivamente al comenzar el bucle

de procesamiento interno de los árboles y detenido de inmediato al finalizarlo. Se excluyó explícitamente de la medición todo el costo computacional asociado a la instanciación de las estructuras, la generación aleatoria de los arreglos y la escritura de datos en disco.

Al finalizar cada simulación, los resultados empíricos fueron exportados a archivos CSV. El formato de estos archivos respetó un encabezado estricto de 17 columnas, documentando métricas críticas como la instancia, la estructura, los contadores totales de transacciones, los éxitos y fallos de cada operación, el tamaño final, la altura alcanzada y el tiempo transcurrido .

3.3. Validación:

Antes de consolidar estos archivos, se aplicó un protocolo de validación de corrección automatizado. Este procedimiento verificó que ambos árboles finalizaran con la misma cantidad de nodos, comprobó una muestra de 100 claves aleatorias para asegurar respuestas idénticas de las estructuras, y escaneó la totalidad del inventario comprobando que la suma matemática del stock disponible y prestado fuesen iguales al stock total registrado, anulando cualquier prueba con saldos negativos .

4. Resultados Empíricos

4.1. Tiempos de Ejecución y Estabilidad:

Las tablas resumen confirmaron un desempeño consistente en ambas estructuras. Los datos demostraron que tanto el árbol binario de búsqueda simple (BST) como el árbol auto-balanceado (RedBlackBST) produjeron exactamente las mismas cantidades de consultas, préstamos y devoluciones exitosas y fallidas a través de todas las instancias, validando la equidad de las secuencias evaluadas.

En relación al tiempo de ejecución, ambas estructuras lograron procesar el volumen máximo de 524,288 operaciones en fracciones de segundo. Sin embargo, en la máxima exigencia, el RedBlackBST demostró una ventaja sistemática promediando 0,239 segundos, superando al BST estándar que promedió 0,271 segundos. Más importante aún, la evaluación de la desviación estándar reveló que la variabilidad de los tiempos de ejecución del árbol rojo-negro fue considerablemente menor. Los histogramas generados reflejan una concentración de los tiempos del RedBlackBST en una banda muy estrecha, evidenciando un comportamiento operativo sumamente estable frente a los tiempos más dispersos del BST.

4.2. Análisis de Alturas Estructurales:

El contraste más marcado que explica esta dinámica se encuentra en la comparación de alturas. El RedBlackBST contuvo su crecimiento estructural promediando una

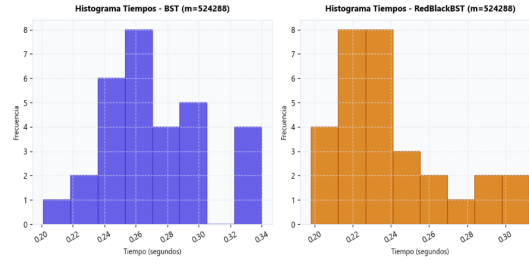


Figura 4: Histogramas de tiempo medición tamaño 524288

altura de apenas 24 niveles en el escenario de mayor volumen. Por el contrario, el árbol binario estándar promedió alturas superiores a 40 niveles, registrando picos de hasta 47 niveles de profundidad. La mayor elongación de las ramas del BST obligó a las operaciones de consulta a recorrer, en promedio, una mayor cantidad de nodos, justificando la penalización observada en su tiempo total y su mayor variabilidad.

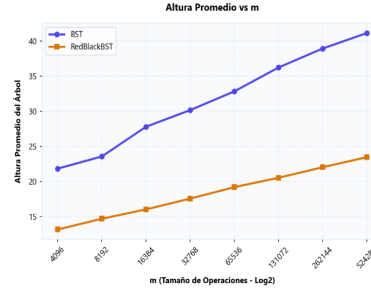


Figura 5: Gráfico de altura promedio

4.3. Crecimiento Asintótico:

Finalmente, al observar el gráfico de dispersión log-log, la correlación entre el tamaño del problema m y el tiempo promedio reveló una tendencia de escalamiento asintótico paralelo para ambas estructuras. El cálculo del ratio de crecimiento demostró que ambas estructuras exhiben un factor multiplicador de tiempo cercano a 2, 3 o 2, 5 al duplicar el tamaño de la muestra. La transformación de este factor mediante $\log_2(\text{ratio})$ produjo valores estabilizados en torno a 1, 2, confirmando visual y matemáticamente que el costo computacional total de la simulación no creció de forma lineal pura, sino que se enmarcó dentro de la complejidad super-lineal esperada.

5. Análisis Teórico

5.1. Tablas de Simbolos:

Las estructuras evaluadas en este laboratorio (BST y RedBlackBST) funcionan bajo el concepto de Tabla de Símbolos (Symbol Table), una estructura de datos abstracta

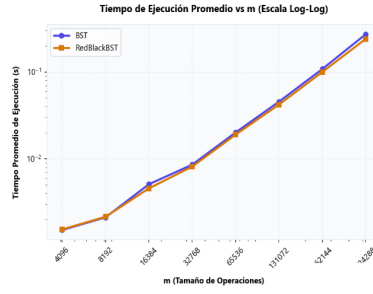


Figura 6: Gráfico log-log de tiempo promedio

que asocia un conjunto de claves con valores. Para este caso, la clave corresponde al identificador numérico del componente y el valor al objeto `InventoryItem`. La eficiencia de la simulación depende del costo de las operaciones de búsqueda y actualización de estas claves.

5.2. Árbol Binario de Búsqueda (BST):

En un árbol binario de búsqueda estándar (BST), el costo de las operaciones es directamente proporcional a la altura del árbol (h). Si los datos se insertan en orden, el árbol se degenera formando una lista enlazada, resultando en una altura N y un tiempo de operación de $O(N)$ en el peor de los casos. Sin embargo, cuando las claves se insertan en un orden aleatorio, el árbol mantiene un balance natural. Matemáticamente, la altura esperada de un BST construido con inserciones aleatorias es proporcional a $4,3 \ln(N)$. Por lo tanto, en el caso promedio, las operaciones toman un tiempo de $O(\log N)$.

5.3. Árbol Rojo-Negro (RedBlackBST):

Por otro lado, el árbol rojo-negro (RedBlackBST) es una estructura auto-balanceada. Mediante la aplicación de reglas de coloración en los enlaces y rotaciones de nodos, garantiza que el camino más largo desde la raíz hasta una hoja nunca sea mayor al doble del camino más corto. Esto acota la altura máxima teórica del árbol a $2 \log_2(N)$. En consecuencia, el RedBlackBST asegura un tiempo de ejecución de $O(\log N)$ para búsquedas e inserciones, incluso en el peor de los casos.

Dado que la simulación procesa una secuencia de m operaciones, y el costo de procesar cada operación individual en ambas estructuras (bajo datos aleatorios) es $O(\log N)$, el tiempo total teórico esperado para ejecutar la simulación completa es de $O(m \log m)$.

6. Comparación vs Teoría

6.1. Crecimiento del tiempo y gráfico log-log:

Los resultados obtenidos en las simulaciones respaldan las predicciones del modelo teórico. Respecto a la complejidad temporal, la teoría establece que el tiempo de ejecución debe crecer según $O(m \log m)$. Para verificar esto, se utilizó la métrica $\log_2(\text{ratio})$ al duplicar el tamaño del problema. Un crecimiento lineal $O(m)$ daría un valor de 1, mientras que un crecimiento cuadrático $O(m^2)$ daría un valor de 2. Los datos muestran que el $\log_2(\text{ratio})$ se estabiliza entre 1,15 y 1,32 en los tamaños mayores. Este valor intermedio confirma que el costo computacional sigue un comportamiento super-lineal logarítmico, coincidiendo con la complejidad teórica esperada de $m \log m$.

6.2. Alturas y su Relación con el Tiempo:

En cuanto a las alturas, el comportamiento de las estructuras también se ajustó a la teoría. El límite teórico para un RedBlackBST con $m = 524,288$ operaciones (asumiendo un máximo de $N \approx m$ nodos) es de aproximadamente 38 niveles. La altura empírica obtenida fue de 24 niveles, cumpliendo la garantía de balanceo. Para el BST simple, el modelo predictivo de $4,3 \ln(N)$ estima una altura cercana a 48 niveles bajo inserciones aleatorias. Los resultados empíricos, que promediaron más de 40 niveles con máximos de 47, demuestran la precisión de esta estimación matemática.

Finalmente, la comparación de tiempos totales entre ambas estructuras refleja el impacto directo de estas alturas. Como las secuencias de operaciones fueron aleatorias, el BST normal nunca enfrentó su peor caso teórico ($O(N)$) y operó bajo el caso promedio ($O(\log N)$), lo que explica por qué sus curvas de tiempo fueron casi idénticas a las del RedBlackBST. La leve diferencia de tiempo a favor del RedBlackBST (0,239s frente a los 0,271s del BST) se justifica por su menor altura (24 frente a 47 niveles), lo que requirió menos comparaciones de nodos durante el procesamiento de las operaciones.

7. Conclusión

El desarrollo de este laboratorio permitió evaluar y comparar el rendimiento de dos estructuras de datos basadas en árboles binarios para la administración dinámica de un inventario. Los resultados demostraron que ambas implementaciones, el BST estándar y el RedBlackBST, son capaces de procesar secuencias masivas de operaciones (hasta 524.288 transacciones) en tiempos inferiores a un segundo, confirmando su viabilidad para problemas de búsqueda y actualización a gran escala.

La principal diferencia observada radicó en la estabilidad de la estructura interna de

los árboles. El RedBlackBST demostró su capacidad de auto-balanceo al limitar su altura máxima a 24 niveles en el escenario de mayor volumen, lo que redujo drásticamente la varianza en sus tiempos de ejecución. Por su parte, el BST dependió exclusivamente de la aleatoriedad de los datos de entrada para no degenerarse, alcanzando alturas de hasta 47 niveles. Esta mayor profundidad obligó al algoritmo a recorrer más nodos, explicando por qué el árbol estándar fue ligeramente más lento y presentó una mayor fluctuación en sus tiempos de respuesta.

Desde la perspectiva algorítmica, la experimentación validó los modelos teóricos de complejidad. El cálculo continuo del $\log_2(\text{ratio})$ en los tiempos de ejecución demostró un crecimiento asintótico asimilable a $O(m \log m)$ para la simulación completa. Además, dado que el diseño experimental inyectó secuencias de operaciones puramente aleatorias, el BST operó siempre bajo su caso promedio teórico de $O(\log N)$ por transacción. Esto impidió que se manifestara su peor caso estructural de $O(N)$, razón por la cual su curva de tiempo total se mantuvo muy cercana a la del árbol balanceado.

En síntesis, si bien un BST simple ofrece un rendimiento logarítmico aceptable cuando se garantiza una distribución probabilística uniforme en la inserción de claves, el RedBlackBST se consolida como la estructura más robusta. Al asegurar un límite matemático estricto sobre su altura y garantizar un tiempo de $O(\log N)$ independiente del orden de entrada de los datos, representa la opción técnica adecuada para sistemas de inventario en entornos de producción.